Republic of the Philippines
CAVITE STATE UNIVERSITY NAIC
(Formerly CAVITE COLLEGE OF FISHERIES)
Bucana Malaki, Naic, Cavite
www.cvsu-naic.edu.ph

INFORMATION TECHNOLOGY DEPARTMENT
Bachelor of Science in Computer Science

COSC 105 – Intelligent Systems

Course Project

Leveraging Advancement in Intelligent Systems in App/Software Development

OVERVIEW

Large Language Models (LLMs) are among the most widely adopted advancements in Intelligent Systems. Through LLM APIs from providers such as OpenAI, Claude, and Gemini, developers can add features like summarization, question answering, planning, and classification to everyday applications. In this project, each group will build and deploy a Progressive Web Application (PWA) whose key functionalities are powered by an existing LLM. A PWA is a web app that can be installed on a device and can work even with limited connectivity. The project provides hands-on experience in integrating AI into modern app and software development.

OBJECTIVES

1. **Research** and read the official documentation of an existing LLM API (OpenAI, Claude, or Gemini).
2. **Design** a Progressive Web Application (PWA) that addresses a real-world need using AI-assisted features.
3. **Integrate** an LLM API into a web application to power its key functionalities.
4. **Practice** responsible AI use, including securing API keys and user data and reviewing AI-generated output.
5. **Demonstrate** the working system through a class presentation and live demonstration.

MATERIALS

- Computer/Laptop
- Web Browser
- Code Editor (VSCode)
- Git and GitHub account
- LLM API key (OpenAI, Claude, or Gemini)
- Database service/account (PostgreSQL, Supabase, Firebase, or MongoDB)
- Hosting accounts (Vercel, Netlify, Firebase, Railway, Render, or Fly.io)
- Presentation software (PowerPoint or equivalent)
- Internet Connection
- LLM documentation references (see Instruction 4)

IMPORTANT NOTES

- Ensure submission within the deadline specified in Google Classroom.
- This is a **Grouped Activity (5 members)**
- This will be presented and demonstrated on the last academic lecture & lab meeting
- **Never expose or commit your API keys** to GitHub or frontend code; store them securely using environment variables.

INSTRUCTIONS

1. **Build a Progressive Web Application (PWA).** Develop a PWA in which some of its functionalities are powered by an existing Large Language Model (LLM) such as OpenAI, Claude, or Gemini. Your PWA must include:
   - **Web app manifest** – the app is installable on a device.
   - **Service worker** – caching and basic offline support (e.g., an offline page or cached content).
   - **Responsive design** – usable on both mobile and desktop screens.
   - **HTTPS hosting** – deployed online with a working hosted link.

2. **Select Your Application Type.** Choose ONE (1) application type for your project. Each type may only be selected by ONE (1) group; therefore, **groups in the same class cannot choose the same application type.** Groups are encouraged to coordinate with one another before finalizing their chosen type.

   Application Types:

   - **Document Summarizer and Q&A** *(Users upload a PDF or paste a document, generate a summary, and ask questions about its contents)*
   - **Personal Finance Insight Tracker** *(Users record income and expenses, set budgets, and ask the AI to explain their spending patterns)*
   - **Language Practice Partner** *(Users practice conversations in English or another selected language with an AI partner that corrects mistakes and explains grammar)*
   - **Recipe and Meal Planning Assistant** *(Users enter available ingredients, food preferences, budget, and meal requirements; the app generates a meal plan and shopping list)*
   - **Research Topic and Literature Organizer** *(A tool for students who need to organize research materials and prepare initial literature-review notes)*
   - **Study Schedule Planner** *(Users enter subjects, examination dates, and available study hours; the app generates a suggested study schedule)*
   - **Project Idea Generator** *(Users select a field, difficulty level, and preferred technology; the app generates project titles, objectives, features, and possible technology stacks)*
   - **Event Planning Assistant** *(Helps student organizations plan seminars, workshops, and campus events)*
   - **Help Desk Ticket Classifier** *(Organizes support requests for a school department, computer laboratory, or small organization)*
   - **Mini Inventory Assistant** *(Helps a small store manage products and generate descriptions or stock reports)*
   - **Travel Packing List Generator** *(Creates personalized packing checklists based on a trip's destination, duration, weather expectations, and planned activities)*

   The functionalities described in parentheses are the **minimum required key functionalities** and must be **functional**, not merely displayed. Additional features are welcome.

3. **Choose Your Tech Stack.** Choose ONE (1) option per layer from the recommended stack below. These options are encouraged; however, groups are free to use other technologies that suit their project.
   - **Frontend:** HTML/JavaScript or TypeScript, Svelte, Vue.js, React
   - **Backend:** Node.js with Express, Python with Flask/FastAPI/Django, PHP with Laravel
   - **Database:** PostgreSQL, Supabase, Firebase, MongoDB
   - **Frontend Deployment:** Vercel, Netlify, Firebase
   - **Backend Deployment:** Railway, Render, Fly.io

4. **Choose Your LLM and Read the Documentation.** Select ONE (1) LLM provider and study its official documentation before integrating it into your app:
    - **OpenAI:** https://developers.openai.com/api/docs/quickstart
    - **Claude:** https://platform.claude.com/docs/en/get-started
    - **Gemini:** https://ai.google.dev/gemini-api/docs/get-started

5. **Present and Demonstrate Your Project.**
    - Create a PowerPoint presentation about your system (see File 1 under Submission Requirements).
    - Demonstrate your project in class, including every key functionality and the LLM-powered features.

SUBMISSION REQUIREMENTS

File 1: Presentation (PPT)

- Type of system chosen
- Key functionalities
- Tech stack overview

File 2: ZIP Folder of the Project

- Complete project source code (frontend and backend)
- Sample environment file (.env.example) without real API keys

Link 1: Hosted Website Link

- Working URL of your deployed PWA

Link 2: GitHub Repository

- Repository link containing the complete source code (no API keys committed)

GRADING RUBRICS

Criteria	Excellent	Good	Satisfactory
Web Design & UI	20 pts	17 pts	14 pts
PWA Functionalities	20 pts	17 pts	14 pts
LLM Integration & Functionality	20 pts	17 pts	14 pts
Presentation & Demonstration	20 pts	17 pts	14 pts
Submission Completeness	20 pts	17 pts	14 pts
Total	100 pts		

Rubric Descriptions

Criteria	Excellent	Good	Satisfactory
Web Design & UI	Clean, consistent, responsive, and easy to navigate	Minor layout or consistency issues	Basic, cluttered, or not responsive
PWA Functionalities	Installable, with service worker, offline support, and HTTPS hosting	Installable; offline support is partial or has minor issues	Missing key PWA features (e.g., not installable)
LLM Integration & Functionality	LLM features and all key functions fully working; API keys secured	Minor bugs or limited LLM features	Several LLM features or key functions incomplete
Presentation & Demonstration	Clear and organized; all functions demonstrated	Minor gaps in explanation or demo	Unclear or incomplete demo
Submission Completeness	All four items complete and accessible	One item incomplete or with minor issues	Two or more items missing or inaccessible

SCORE INTERPRETATION

Score Range	Rating	Interpretation
90–100	Excellent	Exceeds requirements; polished and fully functional
85–89	Very Good	Meets requirements well; minor issues only
80–84	Good	Meets most requirements; some bugs or gaps
75–79	Satisfactory	Meets basic requirements; several areas incomplete
Below 75	Needs Improvement	Requires improvement; major requirements missing